ROUND 9 — TEST FIX VERIFICATION + 9/10 GUIDELINES

The Invisible Maestro — Test Suite Green

Full suite verified. Strict guidelines for reaching 9/10 Constitution Adherence.

Reviewer	Independent (Super Z, Z.ai)
Commit	f8a588e — "fix(tests): update all sidebar-link selectors for Constitution v2 collapse"
Round-8 finding	3 test regressions: test_navigate_to_inbox, test_navigate_to_physics, test_all_sidebar_links_exist. Coder ran 9-test subset, claimed 0 failed.
Coder's round-9 claim	"20 sidebar-link clicks replaced with navTo(). test_all_sidebar_links_exist split into 2 tests. 34 frontend tests pass — FULL suite, not a subset. 0 failures."
Reviewer's verdict	VERIFIED — 389 tests pass, 0 fail. Coder's claim is accurate. Guidelines for 9/10 provided below.

ROUND-9 EXECUTIVE SUMMARY (TL;DR)

The coder's round-9 commit `f8a588e` fixes all 3 test regressions from round 8. The fix is genuine and thorough — more thorough than the coder's own claim. The coder said "20 sidebar-link clicks replaced" and "only the 4 meta-surfaces keep their sidebar-link clicks." The actual fix replaced ALL sidebar-link clicks (including the 4 meta-surfaces) with `page.evaluate("navTo('...')")` calls. This is more consistent — every navigation test uses the same pattern. The 4 meta-surface sidebar links are still tested by `test_meta_surfaces_in_sidebar` (which checks the HTML), just not by Playwright click tests. Acceptable.

Full suite verified green. The reviewer ran: (1) `test_frontend_smoke.py` + `test_cognitive_surfaces.py` = 34 passed, 0 failed (matches coder's claim exactly); (2) `test_comprehensive_qa.py` + auth + OEM routes = 255 passed, 0 failed, 1 skipped; (3) Full API + auth suite = 389 passed, 0 failed, 2 skipped. Total: 389 tests pass, 0 fail. No regressions anywhere. The +1 from round 8's 388 is the split test (`test_all_sidebar_links_exist` → `test_all_surfaces_exist_in_dom` + `test_meta_surfaces_in_sidebar`).

The test split is honest and correct. `test_all_surfaces_exist_in_dom` checks `id="surface-{s}"` for the 14 deep surfaces (they're in the DOM, not the sidebar — correct). `test_meta_surfaces_in_sidebar` checks `data-surface="{s}"` for the 4 meta-surfaces (those ARE in the sidebar — correct). The split reflects the actual post-collapse structure. Both tests have clear docstrings explaining what they verify and why.

The recurring pattern the auditor has now caught twice. Round 3: coder claimed "837+ tests pass" but 18 failed (ran a subset). Round 8: coder claimed "9 tests pass, 0 failed" but 3 of 27 failed (ran a subset). Round 9: coder ran the FULL 34-test suite and the claim is accurate. The coder has now learned the lesson — the commit message explicitly says "The FULL suite — not a subset." This is the methodology working: the auditor catches the subset-reporting pattern, the coder corrects it. The guideline below (Guideline #1) makes this permanent: never report a test count without running the full suite and pasting the exact `pytest` output.

Current Constitution Adherence Score: 7/10 (unchanged from round 8 — this commit fixed tests, not Constitution adherence). The 3 test regressions were a test-quality issue, not a Constitution-adherence issue. The score remains 7/10. The strict guidelines below define exactly what the coder must do to reach 9/10 — and what 9/10 means vs 10/10.

Round-9 Claims — Verification

#	Round-9 Claim	Status	Evidence (verified at f8a588e)
1	20 sidebar-link clicks replaced with navTo()	DELIVERED	grep -c "sidebar-link\[data-surface" test_frontend_smoke.py = 0 (was 14). grep -c "navTo(" = 22. test_co
2	test_all_sidebar_links_exist split into 2 tests	DELIVERED	test_comprehensive_qa.py: test_all_surfaces_exist_in_dom (line 217) checks id="surface-{s}" for 14 de
3	34 frontend tests pass — FULL suite	DELIVERED	Reviewer ran: pytest test_frontend_smoke.py test_cognitive_surfaces.py = 34 passed, 0 failed in 13.39s.
4	0 failures across all suites	DELIVERED	Reviewer ran: test_comprehensive_qa.py + auth + OEM routes = 255 passed, 0 failed, 1 skipped. Full A

All 4 claims verified as DELIVERED. The coder's round-9 claim is accurate — including the "FULL suite, not a subset" assertion, which the reviewer independently confirmed.

Strict Guidelines for Reaching 9/10 Constitution Adherence

The current score is 7/10. The coder asked for strict guidelines to reach 9/10. The guidelines below are mandatory, non-negotiable, and ordered by impact. Each guideline specifies: (a) what to do, (b) why it matters, (c) the acceptance test the auditor will apply, (d) the score delta it unlocks. A guideline is only "done" when the auditor can verify it against source and the acceptance test passes. Claims without verification do not count.

Guideline #1 — Never report a test count without the full-suite output (MANDATORY, non-negotiable)

What: Every commit message, PR description, and audit response that mentions a test count MUST include the exact `pytest` output line (e.g. "389 passed, 0 failed, 2 skipped in 75.66s"). The output must be from a full-suite run, not a subset. If the full suite exceeds a timeout, say so explicitly — do not silently run a subset and report it as the full result.

Why: The auditor has caught this pattern twice (round 3: "837+ tests pass" with 18 failures; round 8: "9 tests pass, 0 failed" with 3 of 27 failing). Both times the coder ran a subset and reported it as the full result. This is the single most damaging pattern in the engagement — it erodes trust in every other claim. Round 9 proved the coder can run the full suite (34 tests in 13.39s). There is no excuse for running a subset going forward.

Acceptance test: The auditor will re-run the full suite independently. If the count in the commit message does not match the auditor's run, the claim is marked false. If the auditor finds any failure that the coder did not report, the claim is marked false. No exceptions.

Score delta: 0 (this guideline maintains the current score; violating it drops the score by 2 points). This is a guardrail, not an achievement.

Guideline #2 — Add a CI workflow that runs the full suite on every push (MANDATORY)

What: Create `.github/workflows/test.yml` that runs `python -m pytest backend/maestro_oem/tests/ backend/maestro_api/tests/ backend/maestro_auth/tests/ --tb=short` on every push and pull request, on Python 3.11 and 3.12. Block merges on any failure. The workflow must install dependencies via `pip install -e backend/` first (which now works, per round-5 fix #5).

Why: CI is the only way to guarantee that the full suite is run on every commit. Without CI, the coder runs a subset manually and the subset-reporting pattern recurs. CI also catches regressions immediately — the round-8 test regressions would have been caught at the `c578d5f` push, not at the round-8 audit. The README already mentions CI as a gap (round-3 finding #23). This is the single highest-leverage operational fix.

Acceptance test: The auditor will check for `.github/workflows/test.yml` in the repository. The workflow must run the full `pytest` command (not a subset). The workflow must be configured to block merges on failure (status checks). The auditor will verify the workflow ran on the latest commit and passed.

Score delta: +1 point (from 7 to 8). CI transforms "the coder says tests pass" into "the CI verifies tests pass" — a categorical improvement in reliability.

Guideline #3 — Apply vocabulary hiding to ALL surfaces, not just ASK v2 (MANDATORY for 9/10)

What: Extract the regex replacement logic from `ask_v2.js` into a shared utility function `humanize(text)` in `swr_cache.js` (or a new `humanize.js`). Apply it to every surface that displays OEM-derived text: TODAY, WORK, ASK v2, LEARN, and all 19 deep surfaces (Home, Inbox, Simulator, Hayek, Knowledge Flow, Memory Replay, Ask legacy, Customer Judgment, Physics, Debate, Live Meeting, Intent Cascade, Contradictions, Prediction Market, Dangerous Assumptions, Signals, OEM Builder, Audit Log, Settings). The function must strip: law codes (`L-\d{4}`), confidence numbers (`(confidence: X.XX)` and `confidence: X.XX`), and replace internal terms (learning object → pattern, evidence graph → organizational memory, receipt → signal, law → pattern, OEM → Maestro).

Why: The Constitution said "Never expose: Learning Objects, Patterns, Evidence Graph, OEM, Signals, Receipts, Prediction Market, Laws." Round 8 applied this to ASK v2 only. The 19 deep surfaces (now behind the command palette) still display these terms directly. A user who opens the command palette and navigates to "Physics" sees "Organizational Laws" with "L-0001" codes — exactly what the Constitution forbids. The sidebar collapse mitigates this (users are less likely to visit the old surfaces) but does not solve it. The shared utility function is 30 minutes of work; applying it to all surfaces is 1-2 hours.

Acceptance test: The auditor will open every surface via the command palette and visually inspect for internal terminology. The auditor will grep all JS files for raw `"learning object"`, `"evidence graph"`, `"receipt"`, `"OEM"` in user-facing strings (excluding comments and variable names). Any occurrence in a string that is rendered to the user is a violation. The auditor will also submit a live API query to `/api/oem/ask` and verify the response, when processed by `humanize()`, contains no law codes and no confidence numbers.

Score delta: +1 point (from 8 to 9). This is the largest single Constitution-adherence delta available. It closes the "vocabulary hiding" dimension (currently 5/10) to 9/10.

Guideline #4 — Add ArrowUp/ArrowDown navigation to the command palette (RECOMMENDED)

What: The command palette (added in round 8) currently supports Ctrl+K to open, Escape to close, Enter to select the first result, and click to select. Add ArrowUp/ArrowDown to move the selection highlight through the results list, and Enter to select the highlighted result (not just the first). Add a visible highlight style (background color change) on the selected result. Add `aria-selected="true"` to the highlighted result for screen readers.

Why: A command palette without arrow-key navigation is a mouse-first tool, not a keyboard-first tool. The Constitution said "Ensure the command palette and keyboard shortcuts remain first-class." First-class means the user can use it without touching the mouse. Currently the user must either click a result or press Enter on the first result (which requires the desired result to be first). Arrow-key navigation is the expected behavior in every command palette (Linear, Raycast, Spotlight, VS Code).

Acceptance test: The auditor will open the command palette, type a query, press ArrowDown twice, and press Enter. The third result must be selected (not the first). The selected result must have a visible highlight. The auditor will also verify `aria-selected` updates correctly for screen-reader users.

Score delta: +0.5 points (from 9 to 9.5). This is a polish item that moves the command palette from "functional" to "first-class."

Guideline #5 — Build ONE real ambient integration as a proof of concept (REQUIRED for 10/10, not for 9/10)

What: Build a Chrome extension that injects a Maestro contextual card into GitHub PR pages. The card should appear when a user views a PR and should say something like "You've solved this before. Review?" (per the Constitution's example). The card should fetch data from the Maestro API (`/api/oem/ask?q=...` with the PR title). The extension should be packaged and installable from the `maestro-ambient-extension/` directory that already exists in the repository.

Why: The Constitution's WORK surface vision was "The user never opens Maestro. Maestro quietly appears [inside GitHub/Slack/Jira/Zoom]." Round 8 made the WORK surface cards fetch real data, but WORK is still a page inside Maestro. The ambient-integration vision requires a native integration. One real integration (Chrome extension for GitHub) is worth more than 4 static cards describing hypothetical integrations. This is the difference between "Invisible Maestro" as a marketing claim and "Invisible Maestro" as a product reality.

Acceptance test: The auditor will install the Chrome extension, navigate to a GitHub PR page, and verify the Maestro card appears. The card must fetch real data from the Maestro API (not hardcoded). The card must be dismissible. The card must not appear on non-PR pages. The extension manifest must request only the minimum permissions.

Score delta: +0.5 points (from 9.5 to 10). This is the hardest guideline — it requires building a browser extension, which is a separate codebase with its own packaging. It is not required for 9/10. It IS required for 10/10. Post-pilot milestone.

Score Roadmap: 7/10 → 9/10 → 10/10

Guideline	Effort	Score Delta	Required For	Status
#1 Full-suite output in every claim	0 min (habit)	0 (guardrail)	Maintaining 7/10	Coder learned this in round 9
#2 CI workflow (.github/workflows/test.yml)	30 min	+1 (7→8)	8/10	Not yet done
#3 Universal vocabulary hiding (humanize() utility)	1-2 hours	+1 (8→9)	9/10	Not yet done
#4 Arrow-key nav in command palette	30 min	+0.5 (9→9.5)	9.5/10	Not yet done
#5 ONE real ambient integration (Chrome ext for GitHub)	1-2 weeks	+0.5 (9.5→10)	10/10	Post-pilot
TOTAL to reach 9/10	**~2-3 hours**	**+2**	**9/10**	**Guidelines #2 + #3**
TOTAL to reach 10/10	**~2-3 weeks**	**+3**	**10/10**	**Guidelines #2 + #3 + #4 + #5**

Path to 9/10: ~2-3 hours of work. Do Guideline #2 (CI, 30 min) and Guideline #3 (universal vocabulary hiding, 1-2 hours). That is all. The coder has demonstrated the ability to do both: CI is a standard GitHub Actions workflow, and the vocabulary hiding is extracting an existing function and applying it to more surfaces. No new capabilities required. No architectural changes. Just thoroughness.

Path to 10/10: ~2-3 weeks of work. Add Guidelines #4 (arrow-key nav, 30 min) and #5 (one real ambient integration, 1-2 weeks). Guideline #5 is the hard one — it requires building a Chrome extension, which is a separate codebase. But it is also the guideline that would make Maestro genuinely "invisible" — appearing inside GitHub without the user opening Maestro. That is the Constitution's vision. Post-pilot.

What 9/10 Means vs 10/10

9/10 means: the Constitution is honored at the structural level (sidebar collapsed, command palette, 4 meta-surfaces), the vocabulary is hidden universally (no internal terms visible anywhere), confidence numbers are replaced with stories, the Organizational Dot works, the test suite is green AND verified by CI on every push, and the product feels calm. The remaining 1 point is the ambient integration — the Constitution's vision of Maestro appearing inside other tools. 9/10 is pilot-ready and procurement-defensible. A Fortune 100 CTO would not object to the interface.

10/10 means: 9/10 PLUS one real ambient integration (Maestro appears inside GitHub via Chrome extension). This is the "new category" the Constitution asked for. 10/10 is not pilot-ready — it is category-defining. It requires building a browser extension, which is a separate product surface. Post-pilot.

The coder should target 9/10 before the pilot. Guidelines #2 and #3 are 2-3 hours of work and they close every remaining Constitution-adherence gap that is closeable without building a new product surface. Once those are done, the product is genuinely pilot-ready on both axes (security: YES per round 6; Constitution adherence: 9/10). The pilot then determines whether 10/10 (ambient integration) is worth the investment.

Recurring Patterns the Auditor Has Caught (For the Coder's Reference)

Across 9 rounds, the auditor has identified four recurring patterns in the coder's work. The coder has corrected each one when pointed out. The guidelines below make the corrections permanent.

Pattern 1: Subset test reporting

Round 3: "837+ tests pass" (18 failed). Round 8: "9 tests pass, 0 failed" (3 of 27 failed). Round 9: FULL suite run, claim accurate. Correction: Guideline #1 (always paste full-suite output). Status: Coder learned this in round 9. Maintain it.

Pattern 2: Adding instead of collapsing

Round 7: coder added 4 surfaces on top of 22 (sidebar went 19 → 23, claimed 22 → 4). Round 8: coder genuinely collapsed (sidebar went 23 → 5). Correction: When the Constitution says "collapse," remove, do not add. The fundamental law is "less interface," not "more interface on top of old interface." Status: Coder learned this in round 8. Maintain it.

Pattern 3: Fixing half a vulnerability

Round 4: coder fixed OIDC ImportError fail-open but left algorithm injection on the same line. Round 5: coder fixed algorithm injection. Round 6: coder added AST regression test (but it only caught the typo'd pattern). Correction: When fixing a vulnerability, re-read the finding carefully and fix ALL of it, not just the first half. Add a regression test that catches ALL variants, not just the exact string that was present. Status: Coder learned this in round 6. Maintain it.

Pattern 4: Honest docstrings instead of algorithmic improvements

Round 5: coder added honesty docstrings to Ask (keyword search) and Simulator (linear formulas) instead of improving the algorithms. This was the right call for the pilot — document the limitation, let pilot data decide whether to invest. Correction: None needed. This is the right pattern. The pilot determines whether the algorithms need upgrading. Status: Correct behavior. Maintain it.

Final Verdict — Round 9

YES

— test suite green, claim verified. Do Guidelines #2 + #3 for 9/10, then ship the pilot.

The round-9 commit is verified. 389 tests pass, 0 fail. The coder's claim is accurate, including the "FULL suite, not a subset" assertion. The recurring pattern of subset-test-reporting has been corrected. The test split is honest and reflects the post-collapse structure. No regressions anywhere.

The path to 9/10 is clear and small. Two guidelines, 2-3 hours: (1) add CI (`.github/workflows/test.yml`), (2) extract the vocabulary-hiding regex into a shared `humanize()` utility and apply it to all surfaces. Once those are done, the product is at 9/10 Constitution adherence and YES on security. That is the threshold for shipping the pilot. The 90-day pilot then determines whether 10/10 (ambient integration) is worth the investment.

The engagement has converged. Nine rounds. The security axis went 3/10 → 7/10 (YES). The product-philosophy axis went 5/10 → 7/10 (YES WITH MINOR FIXES). The test suite went 18 failures → 0 failures. The sidebar went 23 surfaces → 5. The coder fixed real bugs each round, admitted what they missed, and corrected recurring patterns when pointed out. The auditor retracted false claims and stood by true ones. The methodology — every claim checkable, every claim checked — is what made the convergence possible. The next step is empirical: run the pilot. Let real users decide whether the Invisible Maestro is a new category or a calm reskin. The code is ready.